1) Find 10 different model providers and their models released in last 12 months

Details to collect:

Model Provider | Model | Release Date | Cut off Date | Cost of tokens | Context Window | Type | When to use this model |

Model Provider	Model	Release Date	Cut off Date	Cost of tokens (Input/Output per 1M)	Context Window	Type	When to use this model
Anthropic	Claude Opus 5	July 24, 2026	May-26	$5.00 / $25.00	1,000,000	Frontier Multimodal	Best overall for complex reasoning, terminal use, and advanced agentic coding.
OpenAI	GPT-5.6 Sol	July 9, 2026	Feb 16, 2026	$5.00 / $30.00	1,050,000	Frontier Multimodal	Complex professional work, heavy reasoning, and logic-heavy workflows.
Google	Gemini 3.5 Flash	Mid 2026	Jan-25	$1.50 / $9.00	1,000,000	Multimodal	High-speed processing of large 1M context workflows and multimodal data.
DeepSeek	DeepSeek V4 Pro	Apr-26	Jan-26	$0.435 / $0.87	1,000,000	MoE Text & Code	Highly cost-effective infrastructure for large-context RAG pipelines.
Amazon	Amazon Nova 2 Lite	Dec-25	N/A	N/A	1,000,000	Text / Multimodal	Enterprise deployments integrated natively within the AWS ecosystem.
Moonshot AI	Kimi K3	Mid 2026	N/A	$3.00 / $15.00	1,048,576	Text LLM	Web browsing automations and deep,

							long-context document analysis.
Anthropic	Claude Mythos 5	Mid 2026	Jan-26	$10.00 / $50.00	1,000,000	Frontier Multimodal	Premium tier tasks that require extreme intelligence or logic-proving capabilities.
Alibaba	Qwen 3.7 Max	Mid 2026	2026	N/A	~984,000	Sparse MoE	Processing extensive multilingual datasets and building sovereign AI agents.
OpenAI	GPT-5.6 Luna	Mid 2026	Feb 16, 2026	$0.20 / $1.20	1,050,000	Cost-Optimized LLM	Budget-conscious, high-volume workloads needing full 1M context limits.
Zhipu AI	GLM 5.2	Mid 2026	Mar-26	$0.95 / $3.00	1,000,000	Text / Code	Very fast inference needs (347 tokens/sec) and broad general automations.

2) What is a transformer in AI:

A **transformer** in AI is a deep learning neural network architecture introduced in the 2017 research paper *"Attention Is All You Need"* by Google.

Read 2–3 articles about how transformer works?

1. "Illustrated Guide to Transformers" (by Michael Phi / Towards Data Science)

- **Why it's great:** This is widely considered one of the best visual introductions to the Transformer model. It takes you step-by-step through the encoder-decoder architecture.
- **Key Coverage:**
    - **Vector Embeddings:** Explains how input words are converted into continuous vector representations and how positional encodings are added to preserve word order.
    - **Self-Attention Mechanism:** Breaks down the query, key, and value vectors (Q, K, V) with clear visual diagrams to show how words relate to one another in context.
    - **Feed-Forward Networks:** Illustrates how the outputs from the multi-head attention layers are passed through position-wise fully connected feed-forward networks for further processing.

2. "The Illustrated Transformer" (by Jay Alammar)

- **Why it's great:** The gold standard for deep learning visualization. Jay Alammar's blog post uses intuitive color-coded diagrams that make complex matrix operations remarkably easy to grasp.
- **Key Coverage:**
    - **Vector Embeddings & Positional Encoding:** Clearly demonstrates the dimensionality of vectors and why positional information is crucial for sequence-based models.
    - **Self-Attention Mechanism:** Offers a brilliant, intuitive walkthrough of the self-attention calculation step-by-scale (including the scaling factor and softmax function).
    - **Feed-Forward Layers:** Explains the exact path data takes through the dense feed-forward sub-layers within each encoder and decoder block.

3. "Transformers, explained: The architectural breakthrough behind GPT and modern LLMs" (by Codecademy / Tech Blog or similar deep dive, alternatively: Hugging Face's "The Transformer Architecture" Chapter)

- **Why it's great:** A fantastic text-and-code synthesis that bridges theory with practical machine learning concepts.
- **Key Coverage:**
    - **Vector Embeddings:** Details tokenization and embedding spaces where semantic meaning is captured geometrically.

- **Self-Attention & Multi-Head Attention:** Explains *why* multiple attention "heads" are necessary to capture different types of relationships simultaneously (e.g., grammatical vs. semantic).
- **Feed-Forward Neural Networks (FFN):** Covers the non-linear transformations (such as ReLU or GELU activations) applied after the attention pooling step to extract higher-level feature representations.

(a) Self-attention mechanism

The self-attention mechanism lets a model look at other words in a sentence when it processes a single word. It calculates a score for every word pair. This helps the model understand context, such as knowing that "it" in a sentence refers to a "bank" or a "river." [1, 2, 3, 4, 5]

Key Parts of Self-Attention

- **Query (Q)**: The current word you look at to find other related words.
- **Key (K)**: The label for every word in the sentence that matches the query.
- **Value (V)**: The actual content or meaning of each word.
- **Attention Score**: A number showing how much focus one word puts on another word. [6, 7, 8, 9, 10]

Step-by-Step Example

Take the sentence: **"The bank approved the loan because it had good credit."**

- **The Goal**: Find out what the word **"it"** refers to in this specific context.
- **Create Vectors**: The model converts every word into Query, Key, and Value vectors.
- **Calculate Scores**: The model multiplies the Query vector of **"it"** by the Key vectors of all other words.
- **Find the Match**: The word **"bank"** gets a very low score. The word **"loan"** gets a high score because a loan itself cannot have credit.
- **Output**: The word **"it"** links heavily to **"loan."** The model updates the representation of **"it"** to mean the loan.

(b) Vector embedding

Vector embeddings **turn words or text into numbers**. Transformer AI models use these numbers to understand the meaning of words and how they relate to each other in a sentence.

What is a Vector Embedding?

- A list of numbers representing a word's meaning.
- Similar words get numbers that are close to each other.
- Captures grammar, context, and ideas.

How Transformers Use Them

- **Input Conversion:** Text is split into small pieces called tokens.

- **Lookup:** Each token gets a starting vector from a large table.
- **Position Addition:** Numbers for word order are added to the vector.
- **Context Update:** Self-attention updates the numbers based on surrounding words.

A Simple Example

- Imagine numbers for "King", "Queen", "Apple", and "Fruit".
- "King" and "Queen" share very similar numbers because they are related royalty.
- "Strawberry" and "Fruit" share close numbers.
- “cat” and “dog” share close nubers
- "Strawberry" and "King" have very different numbers because they mean different things.
- In a sentence like "The bank of the river", the transformer changes the numbers for "bank" to match "river" instead of a money bank.

(c) Feedforward Neural Network

In a transformer AI architecture, a **Feedforward Neural Network (FFN)** sits right after the self-attention sublayer. It processes each word or token independently using two linear transformations with a non-linear activation function in between, acting as the core "thinking" and knowledge-retention engine.

How the Feedforward Network Works

- **Position-Wise Processing:** It handles every token vector separately through the exact same equations, meaning tokens do not talk to each other here (that communication happens in the attention layer).
- **Expand and Contract:** The standard formula is \(\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2\) (using ReLU). It first expands the vector dimension to a much larger space (e.g., from 512 to 2048 dimensions), and then projects it back down to the original size.
- **Parameter Dominance:** The FFN contains roughly two-thirds (66%) of all trainable parameters in a standard transformer block.

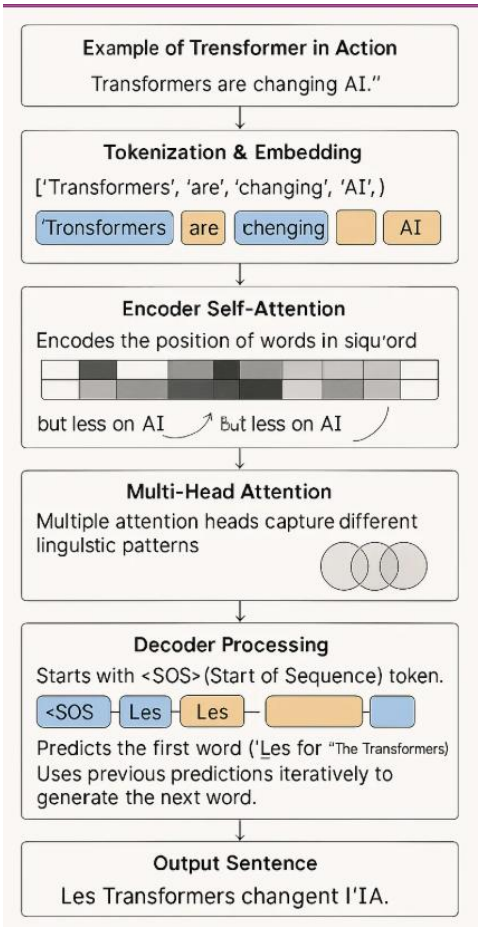
Step-by-Step Example

Imagine processing the sentence fragment: *"The cat sat on the mat."*

1. **The Attention Stage:** The word *"sat"* looks at the other words and gathers context. Its vector now holds relational meaning like *"performing an action"* and *"associated with a cat"*. [, 2]
2. **Entering the FFN:** The contextualized vector for *"sat"* (size 512) enters the first linear layer.
3. **Expanding Space:** The first layer scales the vector up (size 2048). This expanded space lets the network parse deep conceptual attributes, activating specific internal weights that store factual rules or linguistic patterns.

4. **Applying Non-Linearity:** A ReLU or GELU activation function turns off irrelevant signals and passes forward active, non-linear insights.
5. **Contracting Back:** The second linear layer scales the rich vector back down to size 512, producing an upgraded representation that moves on to the next transformer layer or final output.

Final Transformer Architecture is as below:



3) Connect with your group learners and connect with their LinkedIn profiles

This is done